

This will configure the new Ethernet interface on each UML instance. Now, go to the terminal of each UML instance, configure the IP address of eth1 and ping the other instance to see if it communicates. Use *tcpdump* to study packet flow in each instance. You can play around with the route command to route the packets of one interface to the other, try all sorts of *iptables*, *route* and *tcpdump* commands, and make sure things work as needed.

Now, let's look at how to add an Ethernet interface to our instance so that it can talk to the host and send packets over the Internet. For this, start a fresh UML instance—let's say its *umid* is UML. There are various methods to add interfaces that can talk to the Internet in the host. We will discuss the latest method, which is TUN/TAP. It is a virtual interface, which is added to the host that connects the UML instance to the host network. Any packet that UML wants to send to the host or the Internet will send it to this virtual TUN/TAP interface on the host, which will relay it to the host. To add a TUN/TAP based eth0 interface on the UML instance, type the following command on the host terminal:

```
uml_mconsole UML config eth0=tuntap,,192.168.42.44.
```

The IP address provided in the command is the IP address for the tunnel interface. This should be an unused IP address of the subnet on your PC, which is connected to the Internet. My subnet is 192.168.42.0, where 192.168.42.43 is the interface address which is connected to the Internet and 192.168.42.129 is my gateway address. So I have chosen 192.168.42.44 for my UML instance. This command will create an eth0 on your UML instance, which is connected to the tunnel interface. To get it up and running, provide an IP address to the eth0 interface on your UML instance. I have chosen 192.168.42.45. Type the following command on the UML instance terminal to make eth0 start:

```
ifconfig eth0 192.168.42.45 up
```

Typing this command will show you a lot of commands being executed by the UML instance. These are the commands actually executed on your host PC by the *uml_net* helper thread, to make this interface on your UML instance connect to the host network interface (see Reference 2 to understand these commands). Also, after this command, type *ifconfig* on your host, which will show that the tap0 interface has been created on your host, which is the tunnel interface to connect the UML instance to your host. Also, type the *route -n* command on your host, and you will see that a route to your UML IP address is added to the host kernel routing table. Be careful while selecting the IP address for your UML instance and your tunnel instance. You may mess up your kernel routing table and your host network may fail.

You now have a UML network interface that can

connect to your host network. Start one *tcpdump* instance on your host and connect it to the tap0 interface. Start a second *tcpdump* instance, and connect it to your host Internet network interface. Ping your tunnel interface from UML (IP address: 192.168.42.44) and this will work. You can also check packet transfers on *tcpdump*. You can now ping to your host Internet interface from UML (IP address: 192.168.42.43) and it will also work—verify this on your host using *tcpdump*. Now, if you ping some IP address that is external to your host, such as 8.8.8.8, the pings will complain that the network is unreachable. Type the *route -n* command on your UML instance, and you will see there is no default gateway configured on it. So any packet outside the tunnel interface subnet will fail. This is because the only access for the UML network to the outside world is the tunnel interface on the host. Configure this tunnel interface as the default gateway by using the following command:

```
route add default gw 192.168.42.44
```

Now, if you ping to 8.8.8.8, the ping will succeed. Congratulations! Your UML instance is now connected to the Internet. Next, ping your UML instance from the host, and this shows that this instance is like any other physical machine over the Internet. Now if you try to ping *www.google.com*, pings will fail. This is because no *nameserver* is configured on your UML instance. Create a file named */etc/resolv.conf*. Open this file and type the following command and save it:

```
nameserver 8.8.8.8 #google nameserver
```

Now if you ping *www.google.com*, the pings will get a reply. Your UML instance is now configured with a *nameserver* and is connected to the Internet. You can even access your UML instance through the SSH shell from your host. Also, if your UML instance is configured with a global Internet address, you can access it from any PC in the world. You can run HTTP or FTP servers on your UML instance, and access it from the outside world. You can even debug your UML network stack using GDB as was demonstrated earlier. The possibilities for using the UML network feature are vast. 

References

- [1] <https://lx.missinglinkelectronics.com/+trees>
- [2] User Mode Linux – Jeff Dike

By: Santosh Ramroop Yadav

The author is an embedded systems developer at Bitcomm Technologies Pvt Ltd. His areas of interest are Linux development, microcontroller firmware, networking and RTOS. He can be contacted at santosh678yadav@gmail.com.